NUMERICAL COMPUTING II
ASSIGNMENT VIII
Simpson's 1/3 Rule and Simpson's 3/8 Rule

Solutions using the same three problems and data as Assignment VII

Formulas and applicability

Let $h=(b-a)/n$ and $x_i=a+ih$. Simpson's rules approximate the function by quadratic or cubic polynomials over groups of subintervals.

Composite Simpson's 1/3 rule (n must be even):

$$S_{1/3} = (h/3)[f_0 + f_n + 4(f_1+f_3+\cdots+f_{n-1}) + 2(f_2+f_4+\cdots+f_{n-2})]$$

Composite Simpson's 3/8 rule (n must be divisible by 3):

$$S_{3/8} = (3h/8)[f_0 + f_n + 3\Sigma(f_i \text{ for i not divisible by 3}) + 2\Sigma(f_i \text{ for interior i divisible by 3})]$$

Important: the number of subintervals must satisfy each rule's constraint. Therefore, when Assignment VII's n values are incompatible, this report uses the closest valid sets for a fair comparison and explicitly identifies the changes.

Question 1. $I = \int_0^1 x^2\,dx$

The exact value is $I=1/3\approx0.333333333333$.

A. Composite Simpson's 1/3 rule

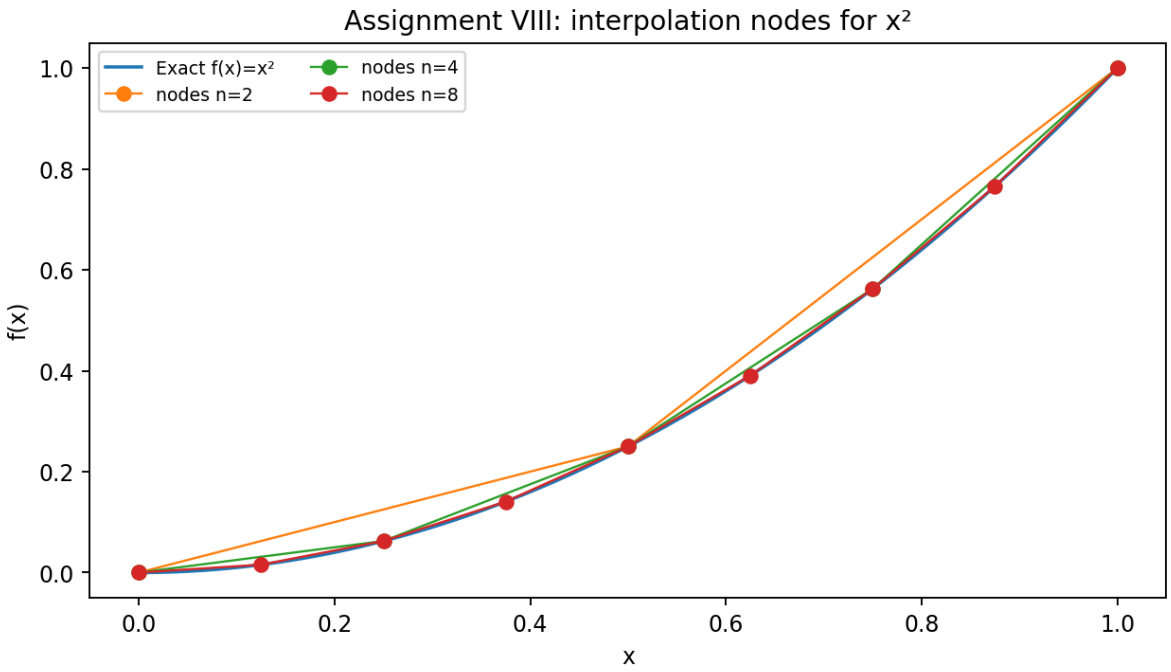
The rule requires even n. The n=1 case from Assignment VII is not valid for composite Simpson's 1/3, so use n=2,4,8,16.

n	h	Simpson 1/3 approximation	Absolute error
2	0.5	0.333333333333	0
4	0.25	0.333333333333	0
8	0.125	0.333333333333	0
16	0.0625	0.333333333333	0

B. Composite Simpson's 3/8 rule

The rule requires n divisible by 3. None of n=1,2,4,8,16 satisfies this condition, so use n=3,6,12,24.

n	h	Simpson 3/8 approximation	Absolute error
3	0.333333333333	0.333333333333	0
6	0.166666666667	0.333333333333	0
12	0.0833333333333	0.333333333333	5.55111512313e-17
24	0.0416666666667	0.333333333333	5.55111512313e-17



Observation: both Simpson rules integrate a quadratic function exactly in exact arithmetic. The reported errors are zero or close to machine precision, unlike the trapezoidal rule's nonzero error for x^2.

Question 2. $I = \int_0^1 e^{(x^2)}\,dx$

A high-accuracy numerical reference value is $I \approx 1.462651745907$. The reference is computed with composite Simpson's 1/3 rule using 200,000 subintervals.

A. Composite Simpson's 1/3 rule

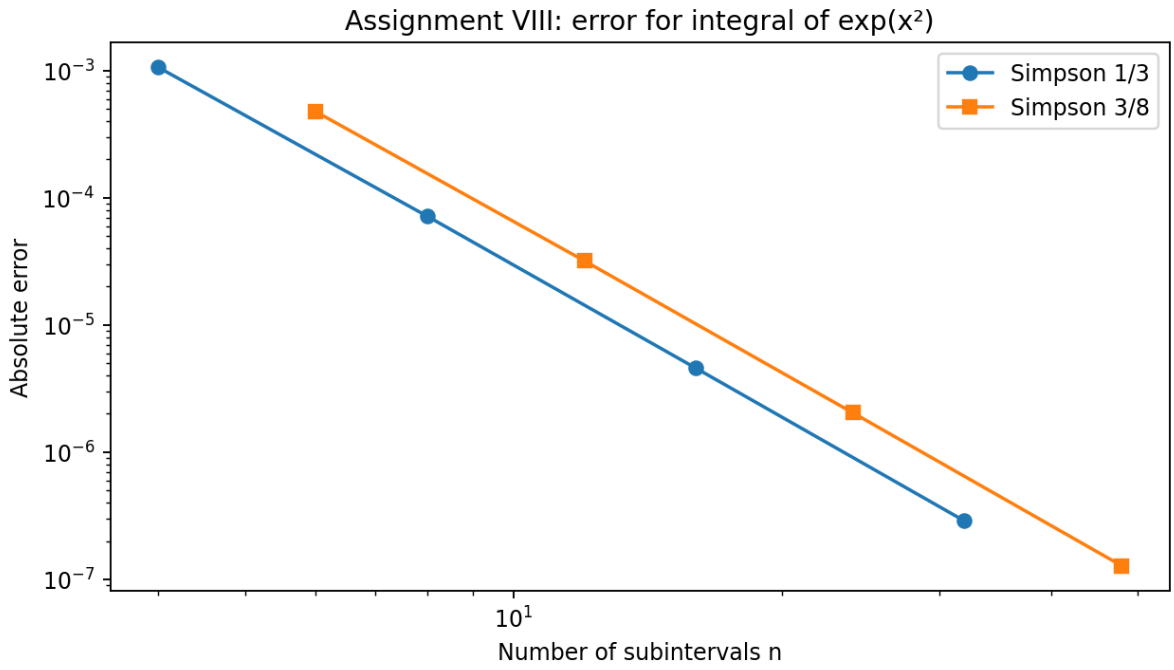
All requested n values 4,8,16,32 are even and therefore valid.

n	h	Simpson 1/3 approximation	Absolute error
4	0.25	1.46371076045	0.00105901453841
8	0.125	1.46272341467	7.16687660869e-05
16	0.0625	1.46265632139	4.57548223909e-06
32	0.03125	1.46265203343	2.87518229003e-07

B. Composite Simpson's 3/8 rule

The requested values 4,8,16,32 are not divisible by 3. To apply Simpson's 3/8 consistently, use n=6,12,24,48.

n	h	Simpson 3/8 approximation	Absolute error
6	0.166666666667	1.46312815796	0.000476412053145
12	0.0833333333333	1.46268369986	3.19539567137e-05
24	0.0416666666667	1.46265378109	2.03518055919e-06
48	0.0208333333333	1.46265187372	1.27811605077e-07



Observation: for this smooth integrand, the errors generally decrease as n increases. Simpson's rules are typically more accurate than the trapezoidal rule for smooth functions because they account for curvature with higher-order polynomial approximations. Increasing n usually improves the result here, but it is not an unconditional guarantee for every function or in the presence of round-off error.

Question 3. Data instead of a function

Use the same experimental data as Assignment VII:

x	**0**	**0.5**	**1.0**	**1.5**	**2.0**
f(x)	1.00	1.65	2.70	4.48	7.39

Here h=0.5 and there are 4 subintervals.

A. Simpson's 1/3 rule

Four is even, so Simpson's 1/3 rule applies directly:

$S_{1/3} = (h/3)[f_0+f_4+4(f_1+f_3)+2f_2]$

$S_{1/3} = (0.5/3)[1.00+7.39+4(1.65+4.48)+2(2.70)] = 6.385000.$

B. Simpson's 3/8 rule

Simpson's 3/8 rule requires a number of subintervals divisible by 3. The full dataset has 4 subintervals, so the rule cannot be applied directly to all four. To estimate the whole integral using the available data, apply Simpson's 3/8 rule over [0,1.5] (the first three subintervals), then the trapezoidal rule over the final interval [1.5,2.0].

Simpson 3/8 on [0,1.5] = 3.474375; trapezoid on [1.5,2.0] = 2.967500.

Combined estimate = 6.441875.

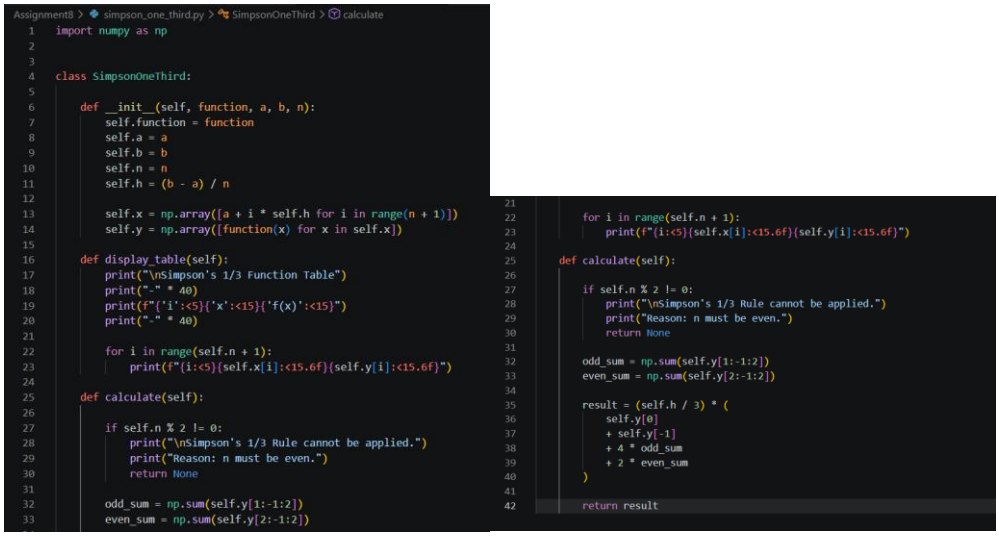
Numerical integration is useful because the data are experimental values rather than a known formula. A numerical rule can estimate area without needing an analytic antiderivative.

Overall conclusion

Simpson's 1/3 rule requires an even number of subintervals; Simpson's 3/8 rule requires a multiple of three. Both are generally higher-order methods than the trapezoidal rule for sufficiently smooth functions. The constraints matter: the rules should not be applied with an invalid n, and experimental data may require a hybrid method or additional measurements.

Python code here:

Simpson 1/3:



```python
import numpy as np


class SimpsonOneThird:

    def __init__(self, function, a, b, n):
        self.function = function
        self.a = a
        self.b = b
        self.n = n
        self.h = (b - a) / n

        self.x = np.array([a + i * self.h for i in range(n + 1)])
        self.y = np.array([function(x) for x in self.x])

    def display_table(self):
        print("\nSimpson's 1/3 Function Table")
        print("-" * 40)
        print(f"{'i':<5}{'x':<15}{'f(x)':<15}")
        print("-" * 40)

        for i in range(self.n + 1):
            print(f"{i:<5}{self.x[i]:<15.6f}{self.y[i]:<15.6f}")

    def calculate(self):

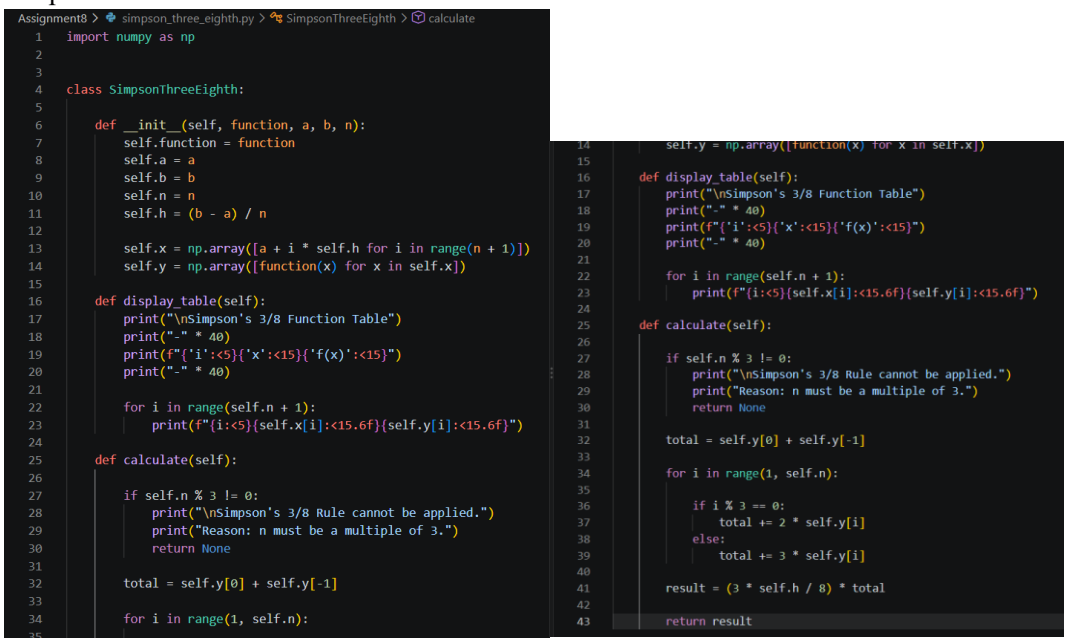
        if self.n % 2 != 0:
            print("\nSimpson's 1/3 Rule cannot be applied.")
            print("Reason: n must be even.")
            return None

        odd_sum = np.sum(self.y[1:-1:2])
        even_sum = np.sum(self.y[2:-1:2])

        result = (self.h / 3) * (
            self.y[0]
            + self.y[-1]
            + 4 * odd_sum
            + 2 * even_sum
        )

        return result
```

Simpson 3/8



```python
import numpy as np


class SimpsonThreeEighth:

    def __init__(self, function, a, b, n):
        self.function = function
        self.a = a
        self.b = b
        self.n = n
        self.h = (b - a) / n

        self.x = np.array([a + i * self.h for i in range(n + 1)])
        self.y = np.array([function(x) for x in self.x])

    def display_table(self):
        print("\nSimpson's 3/8 Function Table")
        print("-" * 40)
        print(f"{'i':<5}{'x':<15}{'f(x)':<15}")
        print("-" * 40)

        for i in range(self.n + 1):
            print(f"{i:<5}{self.x[i]:<15.6f}{self.y[i]:<15.6f}")

    def calculate(self):

        if self.n % 3 != 0:
            print("\nSimpson's 3/8 Rule cannot be applied.")
            print("Reason: n must be a multiple of 3.")
            return None

        total = self.y[0] + self.y[-1]

        for i in range(1, self.n):

            if i % 3 == 0:
                total += 2 * self.y[i]
            else:
                total += 3 * self.y[i]

        result = (3 * self.h / 8) * total

        return result
```